TASK 1

In [1]:
```
import os
import cv2
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
import matplotlib.cm as cm

# Create directory if it does not exist
os.makedirs('images', exist_ok=True)

# Define paths (replace filenames with your own if needed)
path_img1 = os.path.join('images', 'cameraman.png')
path_img2 = os.path.join('images', 'cruyff.png')

# Fallback: create sample test patterns if images are missing
if not os.path.exists(path_img1):
    print("Notice: 'cameraman.tif' not found. Creating a synthetic image..."
    sample1 = (np.indices((256, 256))[0] ^ np.indices((256, 256))[1]).astype
    cv2.imwrite(path_img1, sample1)

if not os.path.exists(path_img2):
    print("Notice: 'lena_gray_256.tif' not found. Creating a synthetic image
    sample2 = (np.indices((256, 256))[0] * 0.5 + np.indices((256, 256))[1] *
    Image.fromarray(sample2).save(path_img2)

print("Setup completed. All dependencies and files are ready.")
```

```
Setup completed. All dependencies and files are ready.
```

TASK 2

a

In [2]:
```
# Task #2: a. Loading/reading an image using OpenCV
img = cv2.imread(path_img1)

# OpenCV loads color as BGR; convert if multi-channel, else render as graysc
if len(img.shape) == 3 and img.shape[2] == 3:
    img_display = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    color_map = None
else:
    img_display = img
    color_map = 'gray'

plt.figure(figsize=(4, 4))
plt.imshow(img_display, cmap=color_map)
plt.title("Loaded with OpenCV")
```

```
plt.axis('off')
plt.show()
```

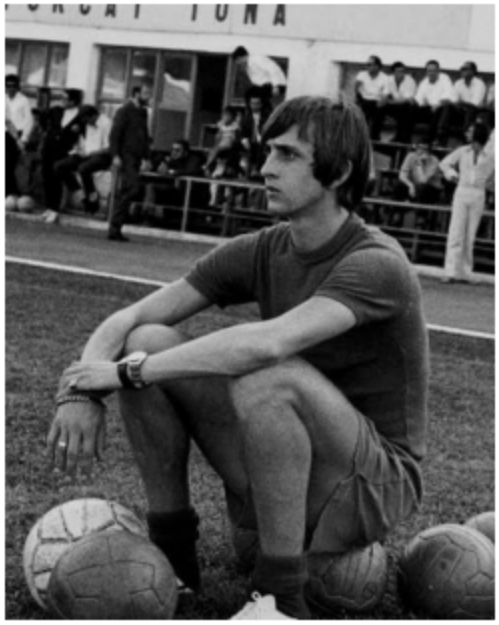


b

In [3]:
```
# Task #2: b. Loading/reading image using PIL
img2 = Image.open(path_img2)

plt.figure(figsize=(4, 4))
plt.imshow(img2, cmap=cm.Greys_r)
plt.title("Loaded with PIL")
plt.axis('off')
plt.show()
```

Loaded with PIL



TASK 3

In [4]:
```
# Task #3: Image Storing
# a. Saving image to disk using OpenCV
cv2.imwrite('new_image.jpg', img)

# b. Saving an image to disk using PIL
img2.save('new_image2.jpg')

print("Files saved successfully: 'new_image.jpg' and 'new_image2.jpg'")
```

```
Files saved successfully: 'new_image.jpg' and 'new_image2.jpg'
```

TASK 4

In [5]:
```
# Task #4: Display image as an Array
print("--- OpenCV Image Array Data ---")
print("Dimensions (Shape):", img.shape)
print("Array Matrix:\n", img)

# Convert PIL Image object to a NumPy array
img_array = np.array(img2)

print("\n--- PIL Image converted to NumPy Array ---")
print("Dimensions (Shape):", img_array.shape)
print("Array Matrix:\n", img_array)
```

```
--- OpenCV Image Array Data ---
Dimensions (Shape): (256, 256, 3)
Array Matrix:
 [[[156 156 156]
  [159 159 159]
  [158 158 158]
  ...
  [151 151 151]
  [152 152 152]
  [152 152 152]]

 [[160 160 160]
  [154 154 154]
  [157 157 157]
  ...
  [154 154 154]
  [155 155 155]
  [153 153 153]]

 [[156 156 156]
  [159 159 159]
  [158 158 158]
  ...
  [151 151 151]
  [152 152 152]
  [152 152 152]]

 ...

 [[114 114 114]
  [132 132 132]
  [123 123 123]
  ...
  [135 135 135]
  [137 137 137]
  [114 114 114]]

 [[121 121 121]
  [126 126 126]
  [130 130 130]
  ...
  [133 133 133]
  [130 130 130]
  [113 113 113]]

 [[121 121 121]
  [126 126 126]
  [130 130 130]
  ...
  [133 133 133]
  [130 130 130]
  [113 113 113]]]

--- PIL Image converted to NumPy Array ---
Dimensions (Shape): (501, 399, 3)
Array Matrix:
```

```
[[[ 65  65  65]
  [ 79  79  79]
  [130 130 130]
  ...
  [208 208 208]
  [208 208 208]
  [208 208 208]]

 [[ 66  66  66]
  [ 91  91  91]
  [147 147 147]
  ...
  [208 208 208]
  [208 208 208]
  [208 208 208]]

 [[ 53  53  53]
  [ 94  94  94]
  [158 158 158]
  ...
  [208 208 208]
  [208 208 208]
  [208 208 208]]

 ...

 [[ 55  55  55]
  [ 42  42  42]
  [ 30  30  30]
  ...
  [ 38  38  38]
  [ 14  14  14]
  [ 10  10  10]]

 [[ 49  49  49]
  [ 39  39  39]
  [ 50  50  50]
  ...
  [ 46  46  46]
  [ 29  29  29]
  [ 40  40  40]]

 [[ 40  40  40]
  [ 31  31  31]
  [ 55  55  55]
  ...
  [ 38  38  38]
  [ 40  40  40]
  [ 72  72  72]]]
```

Assessment - Array Operations with NumPy

In [6]:
```
# Assessment: Operations on the array using NumPy
print("Array Data Type:", img_array.dtype)
print("Minimum Pixel Value:", img_array.min())
print("Maximum Pixel Value:", img_array.max())
print("Mean Intensity Value:", f"{img_array.mean():.2f}")

# 1. Negative / Invert Transformation
inverted_img = 255 - img_array

# 2. Slicing / Center Crop
h, w = img_array.shape[:2]
cropped_img = img_array[h//4 : 3*h//4, w//4 : 3*w//4]

# 3. Binary Thresholding (> 128)
binary_img = np.where(img_array > 128, 255, 0).astype(np.uint8)

# Visualization
fig, axes = plt.subplots(1, 4, figsize=(14, 4))

axes[0].imshow(img_array, cmap='gray')
axes[0].set_title("Original")

axes[1].imshow(inverted_img, cmap='gray')
axes[1].set_title("Inverted (Negative)")

axes[2].imshow(cropped_img, cmap='gray')
axes[2].set_title("Cropped")

axes[3].imshow(binary_img, cmap='gray')
axes[3].set_title("Thresholded (>128)")

for ax in axes:
    ax.axis('off')

plt.tight_layout()
plt.show()
```

```
Array Data Type: uint8
Minimum Pixel Value: 0
Maximum Pixel Value: 236
Mean Intensity Value: 83.36
```

